

ESTUDO DIRIGIDO I

1. TAREFA

A tarefa exercida neste tópico destina-se a primeira atividade do estudo dirigido, contendo os seguintes registros do código C, instruções Assembly, código resultante do depurador (*link to binary*) e binário em hexadecimal correspondente, conforme ilustrado abaixo na tabela da Figura 1.

Código C	Assembly	Debugger	Binário
<code>int main() {</code>	<code>push ebp</code>	<code>push ebp</code>	55
	<code>mov ebp, esp</code>	<code>mov ebp, esp</code>	89 e5
	<code>sub esp, 16</code>	<code>sub ebp, 0x10</code>	83 ec 10
<code>int x, y, z;</code>	<code>mov DWORD PTR [ebp-4], 0</code>	<code>mov DWORD PTR [ebp-0x4], 0x0</code>	c7 45 fc 00 00 00 00
	<code>mov DWORD PTR [ebp-8], 5</code>	<code>mov DWORD PTR [ebp-0x8], 0x5</code>	c7 45 f8 05 00 00 00
	<code>mov DWORD PTR [ebp-12], 9</code>	<code>mov DWORD PTR [ebp-0xc], 0x9</code>	c7 45 f4 09 00 00 00
<code>x = 0;</code>	<code>mov edx, DWORD PTR [ebp-4]</code>	<code>mov edx, DWORD PTR [ebp-0x4]</code>	8b 55 fc
<code>y = 5;</code>	<code>mov eax, DWORD PTR [ebp-8]</code>	<code>mov eax, DWORD PTR [ebp-0x8]</code>	8b 45 f8
<code>z = 9;</code>	<code>add eax, edx</code>	<code>add eax, edx</code>	01 d0
<code>z += x + y + 15;</code>	<code>add eax, 15</code>	<code>add eax, 0xf</code>	83 c0 0f
	<code>add DWORD PTR [ebp-12], eax</code>	<code>add DWORD PTR [ebp-0xc], eax</code>	01 45 f4
	<code>mov eax, 0</code>	<code>mov eax, 0x0</code>	b8 00 00 00 00
<code>return 0;</code>	<code>leave</code>	<code>leave</code>	c9
<code>}</code>	<code>ret</code>	<code>ret</code>	c3

Figura 1. Tabela com código C original, inexistente na Aula 2 (Fonte: Autor)

As instruções Assembly empregadas neste código baseou-se na lista a seguir de comandos mnemônicos:

- push**: salva o valor de um registrador na pilha (*temporary memory*).
- mov**: copia um valor de uma origem para o destino.
- sub**: subtração (usado para reservar espaço na pilha).
- add**: soma (usado para calcular e adicionar constantes).
- leave**: restaura o ponteiro da pilha e o ponteiro de base antes de sair da função.
- ret**: retorna o controle para o ponto onde a função foi chamada.

Além disso, os endereços de memória, que são os locais na pilha onde as variáveis (x, y, z) do código C foram armazenadas, são, respectivamente:

- `[ebp-0x4]`: onde a variável x foi alocada e usada.
- `[ebp-0x8]`: onde a variável y foi alocada e usada.

➤ `[ebp-0xc]`: onde a variável `z` foi alocada e usada.

Para realização dos cálculos e gerenciamento da pilha, os registradores utilizados pela CPU foram, respectivamente:

- ebp (base pointer)**: usado para marcar o início do registro de ativação da função principal (*main*).
- esp (stack pointer)**: usado para gerenciar o topo da pilha com objetivo de reservar espaço.
- eax (accumulator register)**: usado para realizar a soma final e armazenar o resultado que depois é transferido para `z`.
- edx (data register)**: usado temporariamente para segurar o valor da variável `x` durante a soma total.

Por fim, as constantes utilizadas são, respectivamente:

- `0` (ou `0x0`): para inicializar `x` e retornar o sucesso do programa (*return 0*).
- `5` (ou `0x5`): para inicializar `y`.
- `9` (ou `0xc`): para inicializar `z`.
- `15` (ou `0xf`): para a soma final (`add eax, 15`).
- `16` (ou `0x10`): para reservar espaço (*16 bytes*) para as variáveis na pilha.

2. TAREFA

A seguir, a tarefa exercida neste tópico destina-se a segunda atividade do estudo dirigido, conforme ilustrado abaixo na tabela da Figura 2.

Código C	Assembly	Debugger	Binário
<code>int main() {</code>	<code>push ebp</code>	<code>push ebp</code>	<code>55</code>
	<code>mov ebp, esp</code>	<code>mov ebp, esp</code>	<code>89 e5</code>
<code>float a, b, c, d, e;</code>	<code>and esp, -8</code>	<code>and esp, 0xffffffff8</code>	<code>83 e4 f8</code>
	<code>sub esp, 32</code>	<code>sub esp, 0x20</code>	<code>83 ec 20</code>
<code>a = 1.5;</code>	<code>fld DWORD PTR .LC0</code>	<code>fld DWORD PTR ds:0x804a008</code>	<code>d9 05 00 00 00 00</code>
<code>b = 9.8;</code>	<code>fstp DWORD PTR [esp+28]</code>	<code>fstp DWORD PTR [esp+0x1c]</code>	<code>d9 5c 24 1c</code>
<code>c = 0.2;</code>	<code>fld DWORD PTR .LC1</code>	<code>fld DWORD PTR ds:0x804a00c</code>	<code>d9 05 04 00 00 00</code>
<code>d = (b + c + 2.0)/2;</code>	<code>fstp DWORD PTR [esp+24]</code>	<code>fstp DWORD PTR [esp+0x18]</code>	<code>d9 5c 24 18</code>
<code>e = (d * a) - b;</code>	<code>fld DWORD PTR .LC2</code>	<code>fld DWORD PTR ds:0x804a010</code>	<code>d9 05 08 00 00 00</code>
	<code>fstp DWORD PTR [esp+20]</code>	<code>fstp DWORD PTR [esp+0x14]</code>	<code>d9 5c 24 14</code>
	<code>fld DWORD PTR [esp+24]</code>	<code>fld DWORD PTR [esp+0x18]</code>	<code>d9 44 24 18</code>
	<code>fadd DWORD PTR [esp+20]</code>	<code>fadd DWORD PTR [esp+0x14]</code>	<code>d8 44 24 14</code>
<code>return 0;</code>	<code>fld QWORD PTR .LC3</code>	<code>fld QWORD PTR ds:0x804a018</code>	<code>dd 05 10 00 00 00</code>
	<code>fadd st(1), st</code>	<code>fadd st(1), st</code>	<code>de c1</code>
	<code>fld QWORD PTR .LC3</code>	<code>fld QWORD PTR ds:0x804a018</code>	<code>dd 05 10 00 00 00</code>
	<code>fdivp st(1), st</code>	<code>fdivp st(1), st</code>	<code>de f9</code>
	<code>fstp DWORD PTR [esp+16]</code>	<code>fstp DWORD PTR [esp+0x10]</code>	<code>d9 5c 24 10</code>
	<code>fld DWORD PTR [esp+16]</code>	<code>fld DWORD PTR [esp+0x10]</code>	<code>d9 44 24 10</code>
	<code>fmul DWORD PTR [esp+28]</code>	<code>fmul DWORD PTR [esp+0x1c]</code>	<code>d8 4c 24 1c</code>
	<code>fsub DWORD PTR [esp+24]</code>	<code>fsub DWORD PTR [esp+0x18]</code>	<code>d8 64 24 18</code>
	<code>fstp DWORD PTR [esp+12]</code>	<code>fstp DWORD PTR [esp+0xc]</code>	<code>d9 5c 24 0c</code>
	<code>mov eax, 0</code>	<code>mov eax, 0x0</code>	<code>b8 00 00 00 00</code>
	<code>leave</code>	<code>leave</code>	<code>c9</code>
<code>}</code>	<code>ret</code>	<code>ret</code>	<code>c3</code>

	.LC0:				
		.long	1069547520		
	.LC1:				
		.long	1092406477		
	.LC2:				
		.long	1045220557		
	.LC3:				
		.long	0		
		.long	1073741824		

Figura 2. Tabela com código C original, inexistente na Aula 2 (Fonte: Autor)

As instruções Assembly empregadas neste código baseou-se na lista a seguir de comandos mnemônicos, a principal diferença entre esta atividade e a anterior é o uso de registradores e instruções da Unidade de Ponto Flutuante (FPU):

- g. **push**: salva o valor de um registrador na pilha (*temporary memory*).
- h. **mov**: copia um valor de uma origem para o destino.
- i. **and**: usado para alinhamento da pilha, forçando o registrador do topo da pilha a ser múltiplo de 8, isto é, uma técnica de otimização para garantir que os dados estejam em endereços que a CPU processa de forma mais eficiente (*8 bytes para float*).
- j. **sub**: subtração (usado para reservar espaço na pilha).
- k. **fld**: carrega um operando de ponto flutuante para o topo da pilha de registradores da FPU.
- l. **fstp**: armazena o valor do topo da pilha FPU na memória e o remove.
- m. **fadd**: soma (usado para calcular e adicionar valores de ponto flutuante).
- n. **faddp**: soma valores de ponto flutuante e remove o operando da pilha FPU.
- o. **fdivp**: realiza a divisão e remove o divisor da pilha FPU.
- p. **fmul**: multiplica valores de ponto flutuante.
- q. **fsub**: subtrai valores de ponto flutuante.
- r. **leave**: restaura o ponteiro da pilha e o ponteiro de base antes de sair da função.
- s. **ret**: retorna o controle para o ponto onde a função foi chamada.

Além disso, os endereços de memória, que são os locais na pilha onde as variáveis (a, b, c, d, e) do código C foram armazenadas, são, respectivamente:

- **[esp+0x28]**: onde a variável *a* foi alocada e usada.
- **[esp+0x24]**: onde a variável *b* foi alocada e usada.
- **[esp+0x20]**: onde a variável *c* foi alocada e usada.
- **[esp+0x16]**: onde a variável *d* foi alocada e usada.
- **[esp+0xc]**: onde a variável *e* foi alocada e usada.

Para realização dos cálculos especializados em ponto flutuante e gerenciamento da pilha, os registradores utilizados pela CPU foram, respectivamente:

- e. **ebp (base pointer)**: para marcar o início do registro de ativação da função principal (*main*).
- f. **esp (stack pointer)**: para gerenciar o topo da pilha com objetivo de reservar espaço.
- g. **eax (accumulator register)**: para retornar o valor de sucesso (*return 0*).
- h. **st** ou **st(0)**: topo da pilha FPU, é o registrador onde a operação atual (adição, subtração etc.) é realizada.
- i. **st(1)**: segundo elemento na pilha FPU, usado para operações.

Por fim, as constantes utilizadas são, respectivamente:

- 1.5 (1069547520 ou 0x3fc00000): para inicializar *a* e na multiplicação final.
- 9.8 (1092406477 ou 0x411ccccccc): para inicializar *b* e na subtração final.
- 0.2 (1045220557 ou 0x3e4cccccc): para inicializar *c*.
- 2.0 (1073741824 ou 0x40000000): para a soma da expressão da variável *d*.
- 2: divisor da expressão *d*, dividindo por 2.0 que está em .LC3.
- 32 (ou 0x20): para alocar 32 bytes de espaço na pilha.
- 8 (ou 0x8): para alinhamento de memória (*and esp, -8*).
- 0 (ou 0x0): para retornar o sucesso do programa (*return 0*).

3. TAREFA

Por fim, a tarefa exercida neste tópico destina-se a última atividade do estudo dirigido, conforme ilustrado abaixo na tabela da Figura 3.

Código C	Assembly	Debugger	Binário
<code>int main() {</code>	<code>push ebp</code>	<code>push ebp</code>	55
	<code>mov ebp, esp</code>	<code>mov ebp, esp</code>	89 e5
	<code>sub esp, 16</code>	<code>sub ebp, 0x10</code>	83 ec 10
<code>int f, g, h;</code>	<code>mov DWORD PTR [ebp-4], 10</code>	<code>mov DWORD PTR [ebp-0x4], 0xa</code>	c7 45 fc 0a 00 00 00
	<code>mov DWORD PTR [ebp-8], 5</code>	<code>mov DWORD PTR [ebp-0x8], 0x5</code>	c7 45 f8 05 00 00 00
<code>f = 10;</code>	<code>mov eax, DWORD PTR [ebp-4]</code>	<code>mov eax, DWORD PTR [ebp-0x4]</code>	8b 45 fc
<code>g = 5;</code>	<code>and eax, DWORD PTR [ebp-8]</code>	<code>and eax, DWORD PTR [ebp-0x8]</code>	23 45 f8
	<code>mov DWORD PTR [ebp-12], eax</code>	<code>mov DWORD PTR [ebp-0xc], eax</code>	89 45 f4
<code>h = f & g;</code>	<code>mov eax, DWORD PTR [ebp-8]</code>	<code>mov eax, DWORD PTR [ebp-0x8]</code>	8b 45 f8
<code>h = h ^ g;</code>	<code>xor DWORD PTR [ebp-12], eax</code>	<code>xor DWORD PTR [ebp-0xc], eax</code>	31 45 f4
	<code>mov eax, 0</code>	<code>mov eax, 0x0</code>	b8 00 00 00 00
	<code>leave</code>	<code>leave</code>	c9
<code>return 0;</code>	<code>ret</code>	<code>ret</code>	c3
<code>}</code>			

Figura 3. Tabela com código C original, inexistente na Aula 2 (Fonte: Autor)

As instruções Assembly empregadas neste código baseou-se na lista a seguir de comandos mnemônicos:

- a. **push**: salva o valor de um registrador na pilha (*temporary memory*).
- b. **mov**: copia um valor de uma origem para o destino.
- c. **sub**: subtração (usado para reservar espaço na pilha).
- d. **and**: executa a operação lógica E sobre cada bit dos operandos.
- e. **xor**: executa a operação lógica OU exclusivo e é utilizado para complementar bits individuais em registradores ou posições de memória (quando comparado com 1).
- f. **leave**: restaura o ponteiro da pilha e o ponteiro de base antes de sair da função.
- g. **ret**: retorna o controle para o ponto onde a função foi chamada.

Além disso, os endereços de memória, que são os locais na pilha onde as variáveis (f, g, h) do código C foram armazenadas, são, respectivamente:

- `[ebp-0x4]`: onde a variável *f* foi alocada e usada.
- `[ebp-0x8]`: onde a variável *g* foi alocada e usada.
- `[ebp-0xc]`: onde a variável *h* foi alocada e usada.

Para realização dos cálculos e gerenciamento da pilha, os registradores utilizados pela CPU foram, respectivamente:

- j. **ebp (base pointer)**: usado para marcar o início do registro de ativação da função principal (*main*).
- k. **esp (stack pointer)**: usado para gerenciar o topo da pilha com objetivo de reservar espaço.
- l. **eax (accumulator register)**: usado para realizar as operações lógicas e armazenar o resultado que depois é transferido para *h*.

Por fim, as constantes utilizadas são, respectivamente:

- 0 (ou 0x0): para retornar o sucesso do programa (*return 0*).
- 10 (ou 0xa): para inicializar *f*.
- 5 (ou 0x5): para inicializar *g*.
- 16 (ou 0x10): para reservar espaço (16 bytes) para as variáveis na pilha.

4. REFERÊNCIAS BIBLIOGRÁFICAS

- I. Material didático da disciplina disponível no Teams

- II. Material didático do prof. Murilo Bresciani de Carvalho, adaptado pela profa. Carmen Maria Costa de Carvalho, disponível no Teams (Material suplementar/Outras referências)
- III. Portal <https://carlosrafaelgn.com.br/Asm86/>